Quantium Virtual Internship

Retail Strategy & Analytics — Task 1

Customer segment analysis of chip purchasing behaviour, prepared to support a strategic recommendation for the category review.

Prepared by **Fariba Kazi**

Prepared for Julia, Category Manager | Analysis in Python

1. Data preparation & cleaning

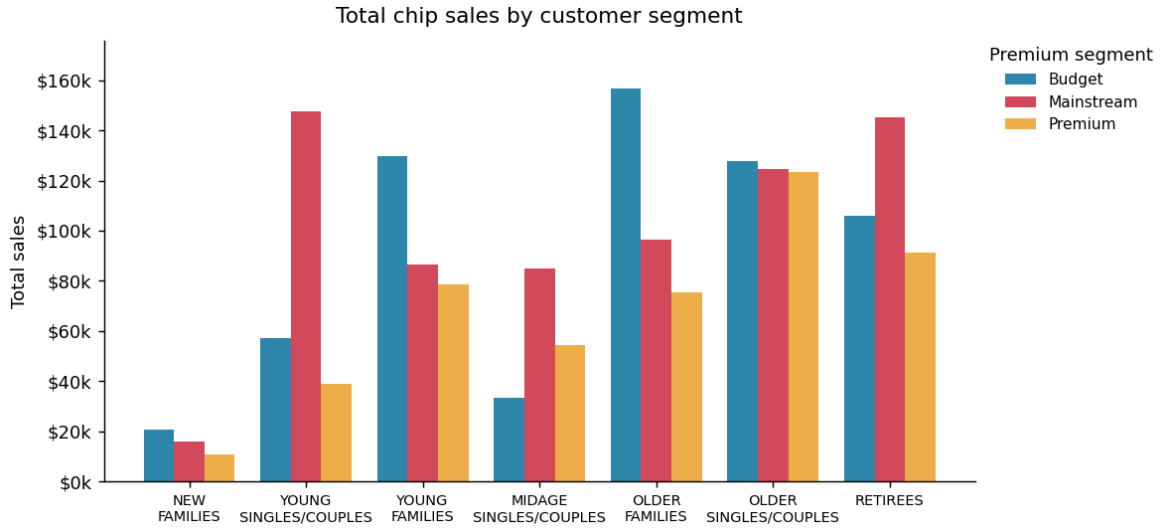
Two datasets were combined: 264,836 chip transactions and 72,637 customer records carrying the LIFESTAGE and PREMIUM_CUSTOMER segment attributes. Before analysis the transaction data was cleaned in four steps:

Step	Action	Result
Date format	DATE was stored as an Excel serial number; converted to calendar dates	2018-07-01 to 2019-06-30
Non-chip items	Salsa products are not chips and were removed from the category	18,094 rows removed
Outliers	One loyalty card bought 200 packs twice — a bulk/commercial buyer, not a retail shopper — and was removed	card 226000 removed
Final dataset	Clean retail chip transactions used for the analysis	246,740 rows

Two new features were engineered from the product name: **PACK_SIZE** (extracted from the weight in grams, ranging 70g–380g) and **BRAND** (the leading word of the product name, with abbreviations such as 'RED'→'RRD' and 'SMITH'→'SMITHS' standardised, giving 20 distinct brands). The transaction and customer tables were then merged on the loyalty card number with no unmatched records.

2. Who spends on chips?

Customers are described by two attributes: **LIFESTAGE** (life position, e.g. young singles, families, retirees) and **PREMIUM_CUSTOMER** (Budget, Mainstream or Premium price/brand preference). Total chip sales were broken down across the 21 segment combinations.



Top segments by total sales:

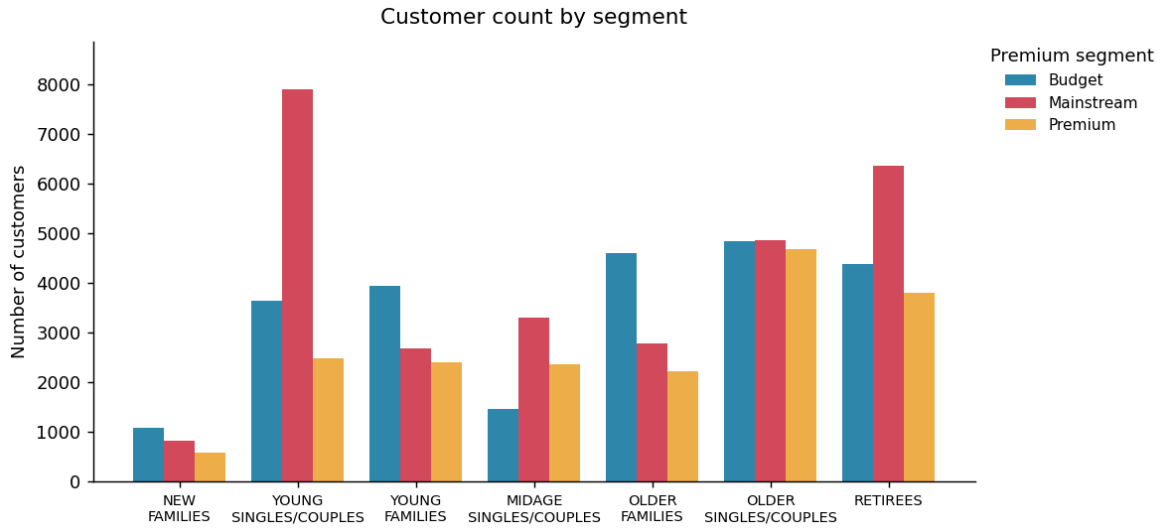
Rank	Lifestage	Premium segment	Total sales
1	Older Families	Budget	$156,864
2	Young Singles/Couples	Mainstream	$147,582

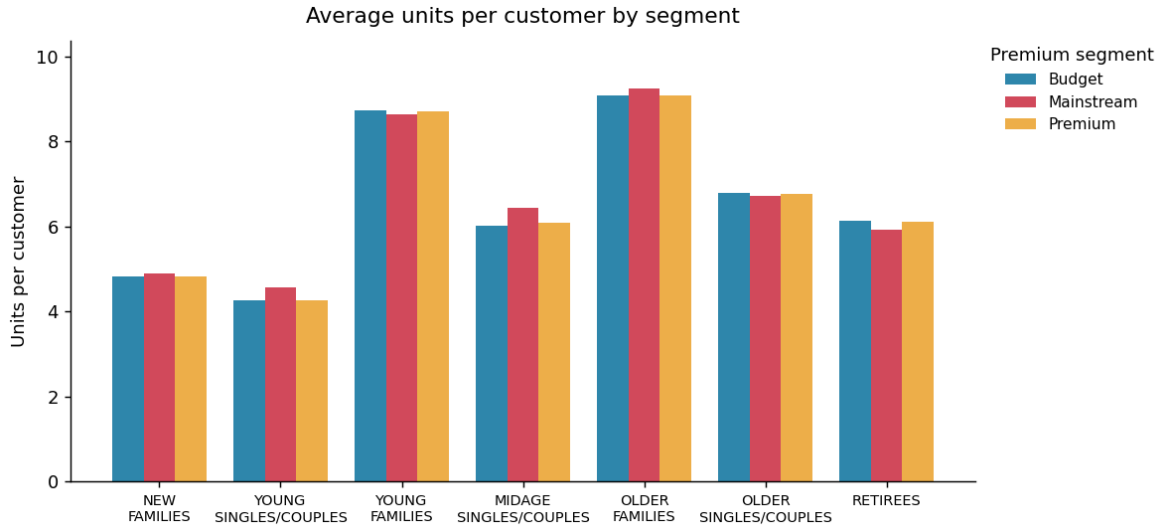
3	Retirees	Mainstream	$145,169
4	Young Families	Budget	$129,718
5	Older Singles/Couples	Budget	$127,834

Sales are dominated by three segments: **Budget – Older Families**, **Mainstream – Young Singles/Couples**, and **Mainstream – Retirees**. The next step is to understand *why* these segments are large — is it more customers, or more spend per customer?

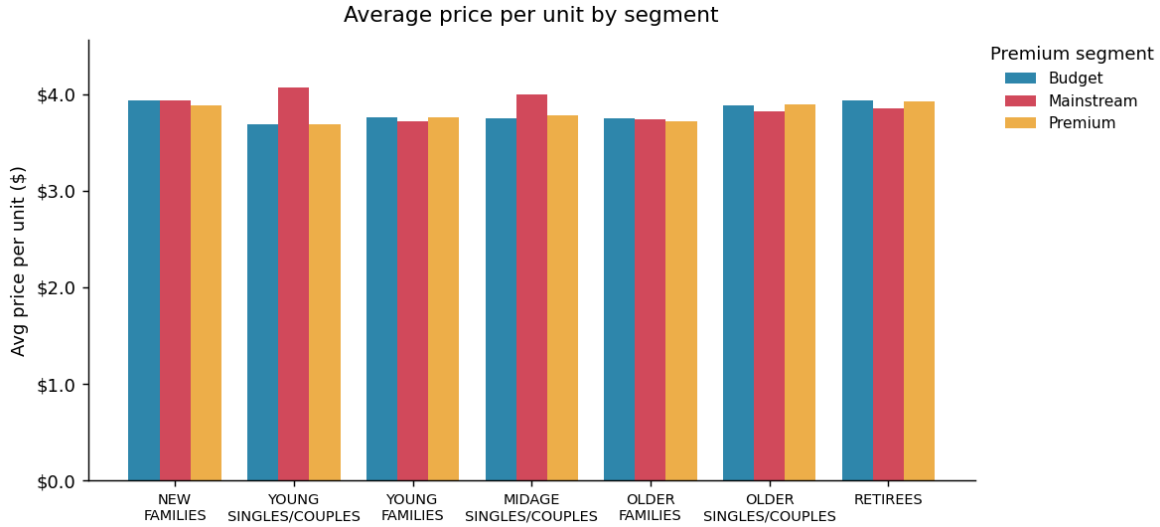
3. What drives the spend?

The large Mainstream – Young Singles/Couples and Retirees segments are driven mainly by having a high **number of customers**. In contrast, the Budget/Young and Older Families segments buy more **units per customer**.





Looking at average price per unit reveals a sharper insight: Mainstream Young and Midage Singles/Couples are willing to pay **more per packet** than Budget or Premium shoppers in the same lifestages.



Average price per unit by segment ($):

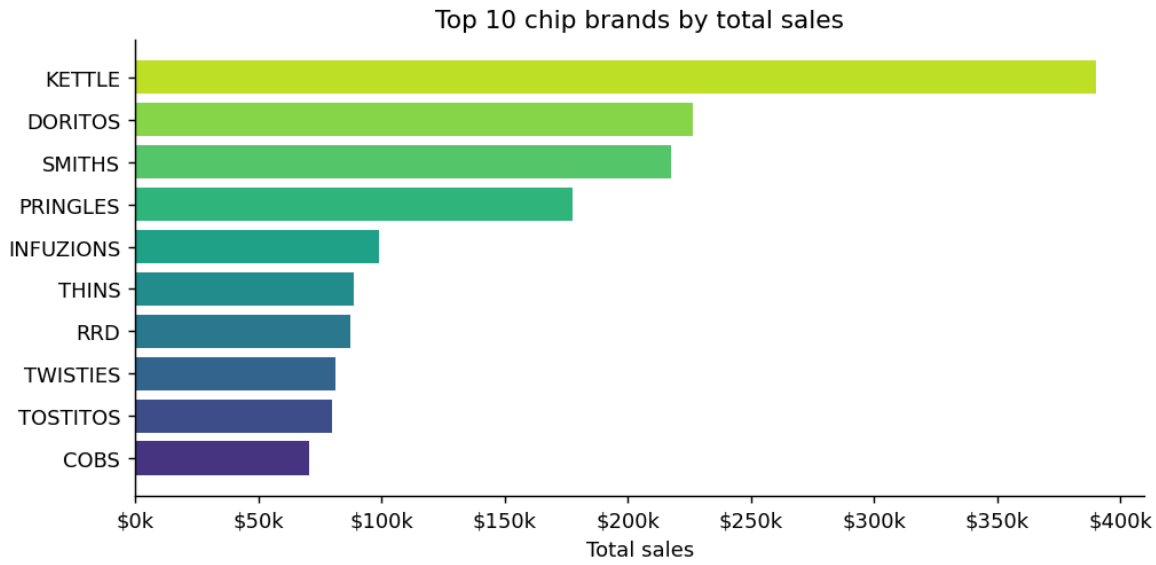
Lifestage	Budget	Mainstream	Premium

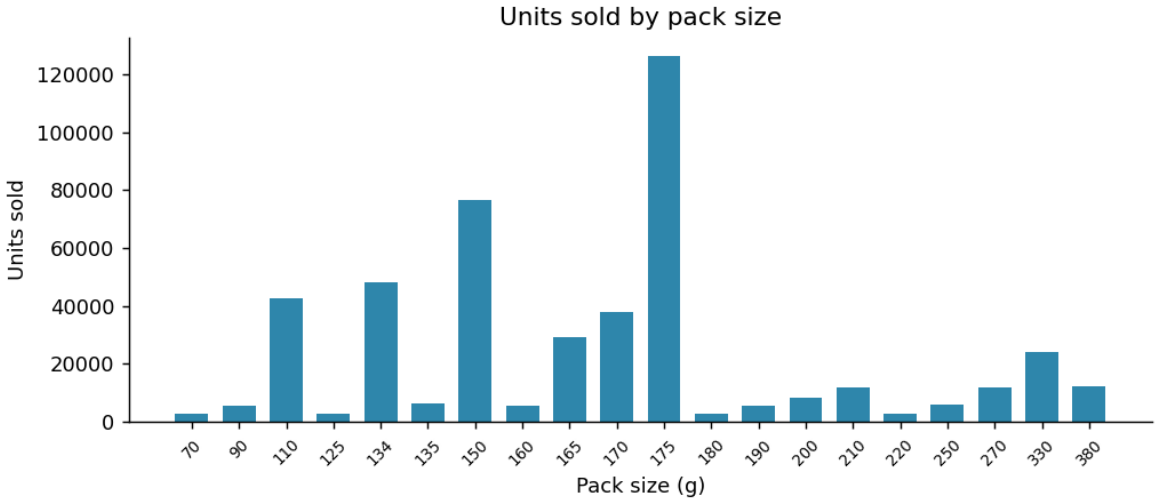
New Families	$3.93	$3.94	$3.89
Young Singles/Couples	$3.69	$4.07	$3.69
Young Families	$3.76	$3.72	$3.76
Midage Singles/Couples	$3.75	$3.99	$3.78
Older Families	$3.75	$3.74	$3.72
Older Singles/Couples	$3.89	$3.82	$3.90
Retirees	$3.93	$3.85	$3.92

The table makes the pattern explicit: within the Young and Midage Singles/Couples lifestages, the **Mainstream** shopper pays the highest price per unit — more than both the Budget and Premium shopper in the same lifestage. A Welch t-test confirms this gap is statistically significant: Mainstream young/midage singles & couples pay an average of $4.04 per unit versus $3.706 for non-mainstream ($t = 37.62$, $p < 0.001$). These shoppers buy chips as an impulse/treat rather than seeking the cheapest option.

4. Brand and pack-size preferences

Kettle is the clear market leader by total sales, ahead of Smiths and Doritos. The most popular pack size by units sold is **175g**, indicating a strong preference for mid-to-large sharing packs.





5. Strategic recommendation for Julia

1. Protect the volume base. Budget – Older Families and Young Families are the largest sources of sales through high units per customer. Maintain competitive pricing and ensure popular large/sharing pack sizes stay well-stocked for these households.

2. Target the premium-willing impulse buyer. Mainstream Young and Midage Singles/Couples are numerous and pay a significant price premium per packet. They are the best audience for higher-margin and on-trend products (e.g. Kettle and other premium ranges), and for off-location/impulse placement and feature space in the category review.

3. Lead with the winning brand and pack format. Anchor the range and promotions around Kettle and the 175g sharing format, which together capture the bulk of category demand.

These segment-level insights give Julia a data-backed basis for range, pricing and space decisions in the upcoming category review.

Appendix — Python code

Full data-prep and analysis script (pandas / scipy / matplotlib).

```
"""
Quantium Virtual Internship - Task 1
Data preparation and customer analytics (Python implementation)
"""
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import os, json

plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 10,
})
OUT = "/home/claude/figs_t1"
os.makedirs(OUT, exist_ok=True)

# colourful but consistent palette
PREMIUM_COLORS = {"Budget": "#2e86ab", "Mainstream": "#d1495b", "Premium": "#edae49"}

# ---------------------------------------------------------------------
# 1. Load
# ---------------------------------------------------------------------
txn = pd.read_excel("/mnt/user-data/uploads/QVI_transaction_data.xlsx")
beh = pd.read_csv("/mnt/user-data/uploads/QVI_purchase_behaviour.csv")

audit = {}
audit["txn_rows_raw"] = len(txn)
audit["beh_rows"] = len(beh)

# ---------------------------------------------------------------------
# 2. Fix data formats
#    DATE is an Excel serial integer -> convert to real date
# ---------------------------------------------------------------------
txn["DATE"] = pd.to_datetime(txn["DATE"], unit="D", origin="1899-12-30")
audit["date_min"] = str(txn["DATE"].min().date())
audit["date_max"] = str(txn["DATE"].max().date())

# ---------------------------------------------------------------------
# 3. Examine products: remove non-chip items (salsa)
# ---------------------------------------------------------------------
audit["unique_products_raw"] = txn["PROD_NAME"].nunique()
is_salsa = txn["PROD_NAME"].str.contains("salsa", case=False)
audit["salsa_rows_removed"] = int(is_salsa.sum())
txn = txn[~is_salsa].copy()

# ---------------------------------------------------------------------
# 4. Outlier check on PROD_QTY
#    A summary reveals one customer buying 200 packs in two transactions
# ---------------------------------------------------------------------
audit["qty_describe"] = txn["PROD_QTY"].describe().round(2).to_dict()
big = txn[txn["PROD_QTY"] >= 200]
audit["qty_200_rows"] = int(len(big))
outlier_cards = big["LYLTY_CARD_NBR"].unique().tolist()
audit["outlier_cards"] = outlier_cards
# This card belongs to a commercial/bulk buyer, not a normal retail shopper -> remove
txn = txn[~txn["LYLTY_CARD_NBR"].isin(outlier_cards)].copy()
audit["txn_rows_clean"] = len(txn)

# ---------------------------------------------------------------------
# 5. Derive features: PACK_SIZE and BRAND from PROD_NAME
# ---------------------------------------------------------------------
txn["PACK_SIZE"] = txn["PROD_NAME"].str.extract(r"(\d+)\s*[gG]").astype(float)
audit["pack_size_min"] = float(txn["PACK_SIZE"].min())
audit["pack_size_max"] = float(txn["PACK_SIZE"].max())

# brand = first word, then tidy up known aliases
txn["BRAND"] = txn["PROD_NAME"].str.strip().str.split().str[0].str.upper()
brand_fix = {
    "RED": "RRD", "SNBTS": "SUNBITES", "INFZNS": "INFUZIONS", "WW": "WOOLWORTHS",
    "SMITH": "SMITHS", "NCC": "NATURAL", "DORITO": "DORITOS", "GRAIN": "GRNWVES",
}
txn["BRAND"] = txn["BRAND"].replace(brand_fix)
audit["n_brands"] = int(txn["BRAND"].nunique())

# ---------------------------------------------------------------------
# 6. Merge with customer behaviour
# ---------------------------------------------------------------------
df = txn.merge(beh, on="LYLTY_CARD_NBR", how="left")
audit["merged_rows"] = len(df)
audit["missing_after_merge"] = int(df["LIFESTAGE"].isna().sum())

# ---------------------------------------------------------------------
# 7. Segment metrics
# ---------------------------------------------------------------------
```

```
seg = df.groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])
segment_summary = seg.agg(
    totalSales=("TOT_SALES", "sum"),
    nCustomers=("LYLTY_CARD_NBR", "nunique"),
    nUnits=("PROD_QTY", "sum"),
    nTxn=("TXN_ID", "nunique"),
).reset_index()
segment_summary["unitsPerCust"] = segment_summary["nUnits"] / segment_summary["nCustomers"]
segment_summary["avgPricePerUnit"] = segment_summary["totalSales"] / segment_summary["nUnits"]
segment_summary["salesPerCust"] = segment_summary["totalSales"] / segment_summary["nCustomers"]

LIFESTAGE_ORDER = ["NEW FAMILIES", "YOUNG SINGLES/COUPLES", "YOUNG FAMILIES",
                   "MIDAGE SINGLES/COUPLES", "OLDER FAMILIES", "OLDER SINGLES/COUPLES",
                   "RETIREES"]

def order_life(s):
    return pd.Categorical(s, categories=LIFESTAGE_ORDER, ordered=True)

# Top segments by total sales
top_sales = segment_summary.sort_values("totalSales", ascending=False).head(5)
audit["top_segments"] = top_sales[["LIFESTAGE","PREMIUM_CUSTOMER","totalSales"]].assign(
    totalSales=lambda d: d.totalSales.round(0)).to_dict("records")

# ---------------------------------------------------------------------
# CHART 1: total sales heat-style grouped bar by segment
# ---------------------------------------------------------------------
def grouped_bar(metric, ylabel, title, fname, money=False, price=False):
    piv = segment_summary.pivot(index="LIFESTAGE", columns="PREMIUM_CUSTOMER", values=metric)
    piv = piv.reindex(LIFESTAGE_ORDER)
    fig, ax = plt.subplots(figsize=(9.2, 4.3))
    x = np.arange(len(piv.index)); w = 0.26
    for i, prem in enumerate(["Budget", "Mainstream", "Premium"]):
        ax.bar(x + (i-1)*w, piv[prem], w, label=prem, color=PREMIUM_COLORS[prem])
    ax.set_xticks(x)
    ax.set_xticklabels([s.replace(" ", "\n") for s in piv.index], fontsize=8)
    ax.set_ylabel(ylabel); ax.set_title(title, pad=10)
    # headroom so bars never reach the top edge
    top = np.nanmax(piv.values)
    ax.set_ylim(0, top * 1.12)
    # legend OUTSIDE the plot on the right so it never overlaps the bars
    ax.legend(frameon=False, fontsize=8.5, title="Premium segment",
              loc="upper left", bbox_to_anchor=(1.01, 1.0), borderaxespad=0)
    if money:
        ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v,_: f"${v/1000:.0f}k"))
    if price:
        ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v,_: f"${v:.1f}"))
    fig.tight_layout(); fig.savefig(f"{OUT}/{fname}", bbox_inches="tight"); plt.close(fig)

grouped_bar("totalSales", "Total sales", "Total chip sales by customer segment",
            "sales_by_segment.png", money=True)
grouped_bar("nCustomers", "Number of customers", "Customer count by segment",
            "customers_by_segment.png")
grouped_bar("unitsPerCust", "Units per customer", "Average units per customer by segment",
            "units_by_segment.png")
grouped_bar("avgPricePerUnit", "Avg price per unit ($)", "Average price per unit by segment",
            "price_by_segment.png", price=True)

# ---------------------------------------------------------------------
# CHART 2: price-per-unit significance (mainstream vs non-mainstream young/midage)
# ---------------------------------------------------------------------
from scipy import stats
target_life = ["YOUNG SINGLES/COUPLES", "MIDAGE SINGLES/COUPLES"]
df["pricePerUnit"] = df["TOT_SALES"] / df["PROD_QTY"]
main = df[(df.LIFESTAGE.isin(target_life)) & (df.PREMIUM_CUSTOMER == "Mainstream")]["pricePerUnit"]
other = df[(df.LIFESTAGE.isin(target_life)) & (df.PREMIUM_CUSTOMER != "Mainstream")]["pricePerUnit"]
tstat, pval = stats.ttest_ind(main, other, equal_var=False)
audit["ttest_price"] = {"t": round(float(tstat),2), "p": float(pval),
                        "main_mean": round(float(main.mean()),3),
                        "other_mean": round(float(other.mean()),3)}

# ---------------------------------------------------------------------
# CHART 3: top brands
# ---------------------------------------------------------------------
brand_sales = df.groupby("BRAND")["TOT_SALES"].sum().sort_values(ascending=False).head(10)
fig, ax = plt.subplots(figsize=(8, 4))
cmap = plt.cm.viridis(np.linspace(0.15, 0.9, len(brand_sales)))
ax.barh(brand_sales.index[::-1], brand_sales.values[::-1], color=cmap)
ax.set_xlabel("Total sales"); ax.set_title("Top 10 chip brands by total sales")
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda v,_: f"${v/1000:.0f}k"))
fig.tight_layout(); fig.savefig(f"{OUT}/top_brands.png", bbox_inches="tight"); plt.close(fig)
audit["top_brand"] = brand_sales.index[0]

# ---------------------------------------------------------------------
# CHART 4: pack size distribution
# ---------------------------------------------------------------------
pack_units = df.groupby("PACK_SIZE")["PROD_QTY"].sum().sort_index()
fig, ax = plt.subplots(figsize=(8, 3.6))
ax.bar(pack_units.index.astype(int).astype(str), pack_units.values,
       color="#2e86ab", width=0.7)
ax.set_xlabel("Pack size (g)"); ax.set_ylabel("Units sold")
ax.set_title("Units sold by pack size")
ax.tick_params(axis="x", labelrotation=45, labelsize=7.5)
fig.tight_layout(); fig.savefig(f"{OUT}/pack_size.png", bbox_inches="tight"); plt.close(fig)
```

```
audit["top_pack_size"] = int(pack_units.idxmax())

# Save segment summary table for the PDF (top 7 by sales)
seg_table = segment_summary.sort_values("totalSales", ascending=False).copy()
seg_table.to_csv("/home/claude/segment_summary.csv", index=False)

with open("/home/claude/audit_t1.json", "w") as f:
    json.dump(audit, f, indent=2, default=str)

print(json.dumps(audit, indent=2, default=str))
print("DONE")
```